

Основные системные вызовы UNIX. `fork()`, `exec()`, `wait()`, `pipe()`, `signal()`

`fork()`

`fork()` — системный вызов, который создаёт новый процесс — процесс-потомок, для которого создаётся собственное адресное пространство и собственные карты трансляции **АДРЕСОВ** (таблицы страниц), но они ссылаются на **СТРАНИЦЫ** адресного пространства предка. При этом для страниц адресного пространства предка права доступа **МЕНЯЮТСЯ** на `only-read` и **УСТАНОВЛИВАЕТСЯ** флаг `copy-on-write`. Если или предок, или потомок попытаются изменить какую-либо страницу, возникнет исключение по правам доступа. Обработывая это исключение, система **ОБНАРУЖИТ** установленный флаг `copy-on-write` и создаст копию страницы в адресном пространстве того процесса, который пытался её изменить. Таким образом код процесса-предка не копируется полностью, а создаются только копии страниц, которые редактируются. Такая ситуация существует до тех пор, пока процесс-потомок не вызовет `exec()` или `exit()`. В результате страницам адресного пространства предка возвращаются обычные права доступа (`read-write`) и флаг `copy-on-write` сбрасывается.

`exec()`

`exec()` — системный вызов, который переводит процесс на адресное пространство программы, которая была передана ему в качестве параметров.

По Вахалии:

Системный вызов `exec` выполняет следующие действия:

1. Разбирает путь к исполняемому файлу и осуществляет доступ к нему.
2. Проверяет, имеет ли вызывающий процесс полномочия на выполнение файла.
3. Читает заголовок и проверяет, что он действительно исполняемый¹.
4. Если для файла установлены биты SUID или SGID, то эффективные идентификаторы UID и GID вызывающего процесса изменяет на UID и GID, соответствующие владельцу файла.
5. Копирует аргументы, передаваемые в `exec`, а также *переменные среды* в пространство ядра, после чего текущее пользовательское пространство готово к уничтожению.
6. Выделяет пространство свопинга для областей данных и стека.
7. Высвобождает старое адресное пространство и связанное с ним пространство свопинга. Если же процесс был создан при помощи `vfork`, производится возврат старого адресного пространства родительскому процессу.
8. Выделяет карты трансляции адресов для нового текста, данных и стека.
9. Устанавливает новое адресное пространство. Если область текста активна (какой-то другой процесс уже выполняет ту же программу), то она будет совместно использоваться с этим процессом. В других случаях пространство должно инициализироваться из выполняемого файла. Процессы в системе UNIX обычно разбиты на страницы, что означает, что каждая страница считывается в память только по мере необходимости.
10. Копирует аргументы и переменные среды обратно в новый стек приложения.
11. Сбрасывает все обработчики сигналов в действия, определенные по умолчанию, так как функции обработчиков сигналов не существуют в новой программе. Сигналы, которые были проигнорированы или заблокированы перед вызовом `exec`, остаются в тех же состояниях.
12. Инициализирует аппаратный контекст. При этом большинство регистров сбрасывается в 0, а указатель команд получает значение точки входа программы.

По Рязани:

Как он выполняется:

1. проверяет, существует ли файл с именем, которое ему передали;
2. проверяет, исполняемый ли он (~~для этого надо прочитать заголовок файла~~);

3. проверяет, есть ли у процесса, который выполняет системный вызов `exec`, права на его запуск;
4. создаётся новое адресное пространство, т.е. создаются таблицы страниц для программы, переданной `exec` в качестве параметра;
5. удалить старые таблицы страниц, указатель на которые содержится в дескрипторе процесса;
6. поменять указатель на новые таблицы страниц;
7. загрузить в регистр `IP` начальный адрес программы (или адрес точки входа программы, или просто точку входа программы), а в регистр `CR3` загрузить адрес таблиц страниц из дескриптора процесса (в дескрипторе мы меняли адрес таблиц страниц, но его ещё надо загрузить в `CR3`), чтобы можно было выполнять страничное преобразование, которое нужно для адресации кода.

Загрузка `CR3` нужна только в процессорах Intel.

Результат: будет выполняться новая программа.

`pipe()`

`pipe()` — системный вызов, который создаёт неименованный программный канал.

Не имеют названия, но имеют дескриптор `inode`.

Чем поддерживаются программные каналы?

Файловой системой.

Где создаётся канал?

Канал создаётся в области данных ядра.

Почему он создаётся там?

Так как адресное пространство процесса **ЗАЩИЩЁННОЕ**, ни один процесс не может обратиться в адресное пространство другого

процесса, единственный способ обмена данными между процессами — через третье адресное пространство.

Для чего используется?

Неименованный программный канал используется для обмена сообщениями (только) между процессами-родственниками, т.к. в результате системного вызова `fork` потомки наследуют от предков дескрипторы открытых файлов, а программный канал — это файл.

Особенности работы

Программные каналы имеют встроенные средства взаимного исключения — массив файловых дескрипторов: из канала нельзя читать, если в него пишут, и в канал нельзя писать, если из него читают.